

Know Your Contract: eIDAS-Based Verifiable Legal Identities for Smart Contracts, Enabling Regulatory-Compliant On-Chain Operations

Awid Vaziry^{✉*}, Sandro Rodriguez Garzon^{✉*}, Christoph Wronka^{✉†}, Axel Küpper^{✉*}

^{*}Service-centric Networking / T-Labs, Technische Universität Berlin, Germany

{vaziry, rodriguezgarzon, axel.kuepper}@tu-berlin.de

[†]Financial Services Audit & Advisory, Baker Tilly, Hamburg, Germany

christoph.wronka@bakertilly.de

Abstract—Public blockchains provide no native mechanism to verify the legal identity behind a deployed smart contract, which blocks institutional adoption and compliance with EU regulations such as MiCA and AMLR. We present *KYC Seal*, the first protocol that extends the EU eIDAS trust infrastructure to Ethereum smart contracts by cryptographically binding them to Qualified Electronic Seals issued by Qualified Trust Service Providers (QTSPs). The protocol realizes the full eIDAS trust chain, from the European Commission’s List of Trusted Lists through Member-State trusted lists and QTSP-signed X.509 certificates down to the individual smart contract, natively on-chain. An on-chain parser extracts identity fields directly from the QTSP-signed certificate bytes at registration. Both cryptographic verifications, the QTSP issuance signature and the certificate holder’s seal signature, are performed once at registration and cached as on-chain state, reducing per-interaction seal verification to a pure state check. A new P-256 elliptic-curve precompile in Ethereum (deployed December 2025) makes these one-time cryptographic steps economical, enabling trustless on-chain verification of eIDAS identities without oracles or runtime intermediaries. A reference implementation, a formal security analysis, and a gas evaluation are the subject of forthcoming work.

Index Terms—Smart Contracts, eIDAS, Qualified Electronic Seal, On-Chain PKI, X.509, Identity, Regulatory Compliance, Ethereum

I. INTRODUCTION

Public blockchain ecosystems execute transactions between counterparties whose legal identities cannot be derived from on-chain data: account addresses are pseudonymous identifiers without verifiable linkage to real-world persons or organisations. While pseudonymity supports privacy and resists censorship, it creates substantial challenges for regulated applications. Traditional Finance (TradFi) institutions and applications involving tokenised Real-World Assets (RWAs) require transactions to be attributable to legally accountable entities [1]. Without such attribution, regulatory compliance and legal certainty cannot be established.

A central obstacle to institutional adoption of public blockchains is the lack of a verifiable trust anchor linking on-chain accounts and smart contracts to real-world legal entities [2]. Financial regulation requires transactions to be attributable, auditable, and traceable to responsible persons or organisations, yet blockchain systems provide no native mechanism for such

attribution [3]. Existing digital identity infrastructures offer no standardised way to project certified organisational identities into decentralised environments. Current on-chain identity approaches rely on off-chain verification, proprietary intermediaries, or unverifiable attestations, resulting in fragmented trust models, increased compliance risk, and single points of failure [2]. Consequently, enterprises lack an interoperable and regulator-recognised method to assert their legal identity within public blockchain ecosystems.

The Ethereum Fusaka upgrade (December 2025) introduced a P-256 precompile (EIP-7951 [4]) that makes cryptography on the eIDAS-mandated curve economical on the EVM for the first time. Building on this tipping point, we present *KYC Seal*, a protocol that extends the EU eIDAS trust infrastructure natively into the Ethereum Virtual Machine (EVM). *KYC Seal* binds smart contracts to Qualified Electronic Seals (QSeal) issued by Qualified Trust Service Providers (QTSPs), realizing the full trust chain from the European Commission’s List of Trusted Lists down to the target contract’s address without runtime reliance on oracles or off-chain intermediaries. An on-chain ASN.1/DER parser extracts identity fields directly from QTSP-signed X.509 certificates, closing the semantic gap between signature verification and identity attribution. Both cryptographic verifications are performed once at registration and cached as on-chain state, reducing per-interaction verification to a pure state check suitable for frequent use by compliance-sensitive contracts.

Our Contribution. We make the following contributions:

- We design *KYC Seal*, the first protocol that extends the EU eIDAS trust infrastructure to EVM smart contracts, enabling trustless on-chain verification of legally recognised identities without oracles or runtime intermediaries.
- We develop the first *on-chain ASN.1/DER parser* for X.509 certificates that extracts public keys, Legal Person Identifiers, and validity periods directly on the EVM, closing the semantic trust gap between QTSP-signed certificates and on-chain identity binding.
- We show that performing both cryptographic verifications once at registration and caching them as on-chain state reduces per-interaction seal verification to a pure state

check with no per-call signature operation.

A reference implementation, a formal security analysis, and an empirical gas evaluation of KYC Seal are the subject of forthcoming work.

Paper Organization. Section II surveys related work on blockchain-based PKI and on-chain organisational identity. Section III introduces the eIDAS trust infrastructure, X.509 certificate structure, and the regulatory drivers for on-chain identity. Section IV defines the system and trust model. Section V presents the KYC Seal protocol. Section VI discusses limitations and future work, and Section VII concludes.

II. RELATED WORK

Blockchain-Based Public Key Infrastructure. Blockchain-based PKI research has followed four threads: smart contracts that automatically impose financial penalties on Certificate Authorities (CAs) that issue unauthorised certificates (IKP [5]), transparency logs recording TLS certificates and revocation on-chain (CertLedger [6], Wang et al. [7]), blockchain-native PKI systems that reimplement issuance, validation, and revocation as smart contracts with CAs operating on-chain (SCPki by Al-Bassam [8], Patsonakis et al. [9], ProofChain by Saleem et al. [10], Yakubov et al. [11]), and zero-knowledge approaches that prove possession of an existing X.509 certificate without revealing its contents (zk-X509 [12]). The first three approaches are *CA-rooted* and extract no identity fields from certificate structures within contract execution [13]; zk-X509 consumes existing PKI but targets natural-person identity with full cryptographic privacy, hiding certificate contents behind an off-chain proof. KYC Seal differs by operating within the eIDAS trust chain, a *PKI-rooted* model whose trust anchor is a multi-authority, government-curated trust-list infrastructure rather than a single CA. It lets Solidity smart contracts verify and consume a legal identity at runtime, which the CA-rooted and zero-knowledge approaches above cannot, establishing legal-person accountability for EU regulatory compliance.

On-Chain Organisational Identity and Trust Frameworks. GLEIF's verifiable Legal Entity Identifier (vLEI) [14] encodes legal-person identity as a wallet-held credential governed by GLEIF rather than as an eIDAS trust service. Bartolomeo [15] developed Functional Credentials, a privacy-preserving construction targeting the EU Digital Identity Wallet for natural persons. Both are off-chain credential formats whose verification occurs between a wallet and a relying party rather than inside a smart contract.

Chainlink's Automated Compliance Engine (ACE)¹ places identity records on public chains, but the underlying vLEI is parsed and verified off-chain by the Chainlink oracle network and only the resulting attestation is written on-chain. The European Blockchain Services Infrastructure (EBSI) [16] operates a permissioned ledger whose trust registries (Trusted Entity Registry, Trusted Accreditation Registry) publish governance-backed issuer and accreditation records, while end-user verifiable credentials are held off-chain in holder wallets under the

W3C self-sovereign model. Both systems move compliance data on-chain, but neither verifies a qualified cryptographic artefact inside contract execution on a public chain: ACE trusts its oracle network, and EBSI confines verification to its own permissioned ledger.

The amended eIDAS regulation introduced the qualified electronic ledger as a new trust-service class (Art. 45k, 45l); Alamillo et al. [17] analyze how such ledgers can provide legal certainty for on-chain records, but the construction presupposes QTSP control of the ledger, positioning it toward permissioned rather than public chains.

KYC Seal (Table I) operates on a genuine eIDAS-qualified artefact (a QTSP-issued X.509 certificate chaining to the EU LOTL), executes full trust-chain verification natively inside an EVM smart contract on a public chain, and binds an individual contract address to a specific legal person. The verified trust chain becomes on-chain state and can be queried within atomic transactions, enabling regulatory-compliant, identified interactions.

III. BACKGROUND

This section introduces the technical and regulatory foundations underlying the KYC Seal protocol: the EU eIDAS trust infrastructure, X.509 certificate structure, and the regulatory drivers for on-chain identity.

Terminology. Throughout this work, we distinguish between *verification* (the cryptographic operation of checking a signature against a public key) and *validation* (the broader process of confirming a certificate's trust chain, revocation status, and policy compliance).

A. EU eIDAS Trust Infrastructure

The eIDAS Regulation (EU No 910/2014) [18], operational since 2016 and amended in 2024 [19], provides the EU's trust framework for electronic identification and trust services. Central to this framework are QTSPs, authorised to issue legally recognised trust services including qualified electronic signatures, seals, and attestations of attributes [18]. Member States publish machine-processable Trusted Lists (TSLs) of QTSPs, aggregated by the European Commission (EC) into the List of Trusted Lists (LOTL) [18]. This infrastructure guarantees that certificates can be traced back to a root controlled by the EC, with revocation and status information available in machine-processable form. A qualified electronic seal is the eIDAS term for a digital signature issued by a legal person. For a seal to qualify, it must be created using cryptographic mechanisms that comply with the EC's implementing regulations. The regulatory reference chain begins with implementing regulation 2024/2979 [20], which specifies ETSI standards for seal formats [21]. ETSI TS 119 312 defines permitted cryptographic suites including the Elliptic Curve Digital Signature Algorithm (ECDSA) with P-256 (secp256r1) as a recommended algorithm.

B. X.509 Certificate Structure

QTSP-issued qualified certificates for electronic seals follow the X.509v3 standard [22], encoded in ASN.1 Distinguished

¹<https://blog.chain.link/automated-compliance-engine-technical-overview/>

TABLE I

POSITIONING OF KYC SEAL AGAINST BLOCKCHAIN-BASED PKI AND ON-CHAIN IDENTITY APPROACHES. **ON-CHAIN IDENTITY** (THE EXTENT TO WHICH CERTIFICATE FIELDS ARE VERIFIED ON-CHAIN): NONE (NO BLOCKCHAIN USED), OFF-CHAIN (VERIFICATION OFF-CHAIN, RESULT ON-CHAIN), PERMISSIONED (YES, BUT ON A RESTRICTED LEDGER), SELECTIVE (ZKP / SELECTIVE DISCLOSURE), YES (FULL PUBLIC-CHAIN VERIFICATION). **COMPOSABLE** (WHETHER ANOTHER CONTRACT CAN ATOMICALLY USE THE TRUSTED IDENTITY DATA DURING EXECUTION): RESTRICTED (PERMISSIONS APPLY), ATTESTATION (COMPLIANCE FLAG ONLY, NO IDENTITY DATA), YES (FULLY QUERYABLE AT RUNTIME).

Approach	Trust model	Intermediary	Regulation	On-chain identity	Composable
IKP [5]	CA-oversight	–	–	Off-chain	No
CertLedger [6]	CA-rooted	–	–	Off-chain	No
SCPki [8]	Web-of-trust	–	–	Off-chain	No
zk-X509 [12]	CA-rooted (ZK)	–	–	Selective	Hashes only
vLEI [14]	GLEIF-rooted	Wallet	ISO 17442	None	No
Funct. Cred. [15]	Wallet-based	–	eIDAS	None	No
Chainlink ACE	Oracle	Chainlink	–	Off-chain	Attestation
EBSI [16]	Governmental	MS validators	eIDAS (impl.)	Permissioned	Restricted
Qual. ledger [17]	QTSP-controlled	QTSP	eIDAS 45k/45l	Permissioned	Restricted
KYC Seal	PKI-rooted	–	eIDAS	Yes	Yes

Encoding Rules (DER). Each certificate contains a To-Be-Signed (TBS) Certificate structure holding the subject’s public key, Legal Person Identifier (LPID) [23], validity period, and issuer information. The QTSP signs this TBS structure to produce the complete certificate. For on-chain verification, two operations are relevant: (i) parsing the TBS certificate to extract the public key and identity fields, and (ii) verifying the QTSP’s signature over the TBS certificate against the QTSP’s own certificate from the TSL.

C. Regulatory Drivers for On-Chain Identity

EU financial regulation requires that value-bearing digital actions be attributable to verifiable legal entities. The Payment Services Directive (PSD2) [24] and its successor, the Payment Services Regulation (PSR) [25], mandate unambiguous attribution of payment instructions to legally responsible persons. The Anti-Money Laundering Regulation (AMLR) [26] requires traceability of both initiating and intermediary parties. The Markets in Crypto-Assets Regulation (MiCA) [27] requires crypto-asset service providers to identify customers, ensure transaction traceability, and maintain auditability. Together, these regulations create an *attribution gap* on public blockchains: pseudonymous on-chain components can perform regulated actions, but there is no native way to verify which legal entity controls them [3].

IV. SYSTEM AND TRUST MODEL

Figure 1 shows the architecture of KYC Seal, which extends the EU eIDAS trust chain onto the blockchain and enables validated on-chain operation without external oracles at runtime. The main trust assumption is the mirroring of the LOTL into the on-chain *TrustedList*, both of which are governance-curated. All subsequent artifacts and operations are verified and validated against this mirrored trust chain, with *TrustedList* as the root of trust. During the registration of a new qualified certificate or qualified seal, it becomes trusted once verified

against the TL during transaction execution and is then persisted in its trusted form on-chain until revocation.

A. System Model

The system model comprises three main smart contracts, three cryptographic artifacts, and five actors, connected by the trust chain of Figure 1. The remainder of this section describes each element.

Contracts and Artifacts. The main contracts are *TrustedList*, which serves as the root trust store for qualified trust services; *SealCertificateRegistry*, which stores and manages the sealing certificate and its issued seals; and *SealedContract*, a base contract designed to be inherited by any contract that requires sealing. The three cryptographic artifacts are the QTSP’s service digital identity (SDI, clause 5.5.3 of ETSI TS 119 612), the issuer-side certificate rooted in the LOTL that issues end-entity seal certificates; the qualified certificate for electronic seal (QSealC, clause 3.3 of ETSI TS 119 495), the end-entity certificate bound to the legal person per eIDAS Art. 3(29)–(30); and the QSeal itself, represented by its TBS data and additional fields extracted from the CAdES envelope and committed raw on-chain.

Actors. Five actors participate: the *Trust Root Governance* curates the on-chain trust store; *QTSPs* sign certificates off-chain under eIDAS vetting; the *Contract Owner* approves a specific *certId* for sealing via *onlyOwner*; the *Certificate Holder* signs the seal payload off-chain with the QSealC private key; and the *Verifier* calls *verifySeal* without any trust requirement. Seal registration is authorised cryptographically by the Certificate Holder’s P-256 signature over the seal payload rather than by *msg.sender*; transaction submitters are therefore not a distinct trust principal.

V. KYC SEAL PROTOCOL

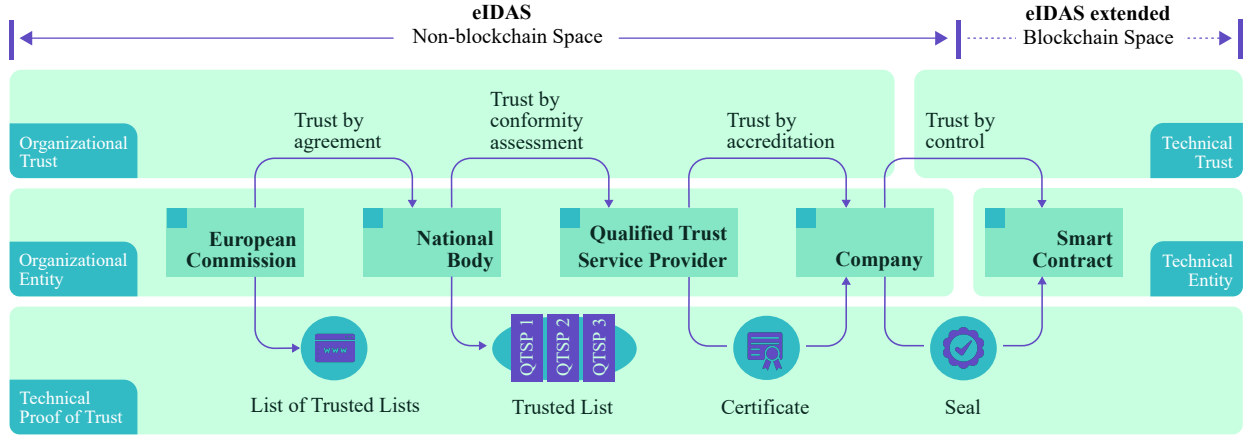


Fig. 1. Layered model of the EU eIDAS trust chain, showing which entities are involved, why those entities are trusted, and how the trust is realized by technical means. The trust chain extension to the blockchain is then illustrated.

Algorithm 1 `registerCertificate(fullCertDER, tsld)`

Require: DER-encoded `QSealC` `fullCertDER`; trust service identifier `tsld`

Ensure: Certificate record stored; `certId` returned

Step 1: Preconditions and identifier derivation

- 1: **require** `ISVALIDTRUSTSERVICE(tsld)`
- 2: `rawTBS` $\leftarrow$ `EXTRACTTBS(fullCertDER)`
- 3: `certId` $\leftarrow$ `sha256(rawTBS)`
- 4: **require** $\neg \text{EXISTS}(\text{certId})$

Step 2: Verify QTSP issuer signature (V2)

- 5: $(r_{\text{cert}}, s_{\text{cert}}) \leftarrow \text{EXTRACTSIGNATURES}(\text{fullCertDER})$

- 6: `scert` $\leftarrow$ `NORMALIZELOWS(scert)`

- 7: $(\text{ts}_x, \text{ts}_y) \leftarrow \text{GETTRUSTSERVICEKEY}(\text{tsld})$

- 8: **require** `P256.verify(certId, rcert, scert, tsx, tsy)`

Step 3: Parse TBS and persist record

- 9: $(\text{pk}_x, \text{pk}_y, \text{lpid}, \text{t}_{\min}, \text{t}_{\max}, \text{serial}) \leftarrow \text{PARSECERTTBS}(\text{rawTBS})$

- 10: **require** `tmin ≤ block.timestamp ≤ tmax`

- 11: Store $(\text{pk}_x, \text{pk}_y, \text{lpid}, \text{t}_{\min}, \text{t}_{\max}, \text{tsld}, \text{serial}, \text{true})$ under `certId`

- 12: **emit** `CERTIFICATEREGISTERED(certId, lpid, tsld)`

- 13: **return** `certId`

This section specifies the *KYC Seal* protocol, which extends the eIDAS trust onto the blockchain, as discussed in Section IV-A. The protocol decomposes into two phases. **Phase 1 (Registration)** establishes the trust chain in on-chain state through five one-time steps: (1.1) trust anchor provisioning, (1.2) certificate registration, (1.3) seal approval, (1.4) off-chain sealing, and (1.5) on-chain seal registration. Their cost is incurred once and then reused: trust anchor provisioning (1.1) and certificate registration (1.2) amortise over all downstream seals, whereas seal approval, sealing, and registration (1.3–1.5) run once per sealed contract. **Phase 2 (Usage)** is the recurring read path: a single step, (2.1) seal verification, re-evaluated on every guarded interaction. Because both P-256 signature

verifications (V1, the certificate holder’s seal signature; V2, the QTSP issuance signature) are performed during Phase 1 and cached as on-chain state, Phase 2 reduces to a pure state check with no per-interaction P-256 operation. The protocol binds a legal identity, vetted by EU regulation, to an on-chain smart contract at address `target` with the chain ID `cid`.

A. Trust Anchor and Certificate Registration

The trust anchor SDI certificate has a 1 to n relationship to `QSealC`, which itself has a 1 to n relationship to issued Seals. The computational and storage costs for the following registration steps can therefore be amortised over various child certificates and Seals respectively.

Step 1.1 (Trust Anchor Provisioning). Making the trusted QTSP certificate of the LOTL available on-chain in the `TrustedList`, which acts as the on-chain root trust store. The Trust Root Governance actor (for example, an EU institution or an Ethereum Foundation governance body) mirrors the full DER-encoded SDI of each relevant QTSP from the web-published LOTL into `TrustedList` via `registerTrustService`. Thereby the TBS is parsed to extract the public key of the QTSP, validity window, and organisational identifier, indexed by `tsld = sha256(rawTBS)` to prevent re-registration. No issuer signature is verified, since trust is governance-curated, just as browser and operating-system root stores ship pre-vetted CA certificates that applications trust implicitly.

Step 1.2 (Certificate Registration). Any party may submit an end-entity `QSealC` certificate for registration. The registry contract first extracts the `TBSCertificate` and derives its identifier `certId = sha256(rawTBS)`. It then verifies the QTSP issuance signature (V2) using the public key of the issuing SDI already stored in `TrustedList`. Only if this signature is valid does it parse the identity fields from the signed TBS and store the certificate record on-chain, as detailed in Algorithm 1.

The QTSP issuance signature on a qualified seal certificate is ECDSA over `sha256(TBSCertificate)` [21]. Hence the `P256.verify` **require** in Step 2 verifies $(r_{\text{cert}}, s_{\text{cert}})$ against

$\text{certId} = \text{sha256}(\text{rawTBS})$, which is equivalent to verifying the signature over the submitted TBS bytes. This matches the X.509/ETSI signature semantics and introduces no additional hashing step beyond the standard certificate digest.

By direct inspection of Algorithm 1, a returned certId implies that tsId is active and that the QTSP issuance signature verifies for $\text{certId} = \text{sha256}(\text{rawTBS})$ under the SDI public key (the two `require` statements). The public key, LPID, validity window, and serial are then parsed from the *same* rawTBS bytes, so no identity field is accepted as independent caller input; under correct parsing, the stored record is bound to the very `TBSCertificate` authenticated by the QTSP signature.

B. Seal Approval, Off-Chain Sealing, and Registration

Step 1.3 (Seal Approval). Before a Seal is registered, the target contract, which inherited `SealedContract`, must pre-approve sealing by giving consent to seal registration by a specific `QSealC`. The Contract Owner of a `SealedContract` calls `approveCertificateForSealing(certId)`, which validates the full certificate trust chain at approval time and records $\text{sealApprovals}[\text{target}] = \text{certId}$ in `SealCertificateRegistry`.

Step 1.4 (Off-Chain Sealing). The Certificate Holder assembles the CAdES envelope [28] carrying the sealed data, computes $\text{envelopeHash} = \text{keccak256}(\text{cadesBlob})$, and signs the seal payload with the `QSealC` private key off-chain, producing (r_s, s_s) . The domain separator provides protocol versioning analogous to EIP-712. With $\text{domSep} = \text{keccak256}(\text{"KYCSeal-v1"})$, the seal payload is:

$$\text{payload} = \text{keccak256}(\text{abi.encode}(\text{domSep}, \text{target}, \text{cid}, \text{cert.lpid}, \text{envelopeHash})). \quad (1)$$

Binding envelopeHash into the V1 payload cryptographically commits the holder to a specific CAdES envelope: any later registration that substitutes a different envelope fails V1 verification, reducing envelope-substitution to an ECDSA forgery against cert.pk . Rotating the envelope (for example, a $\text{BES} \rightarrow \text{LT}$ upgrade) therefore requires a fresh signature and a fresh `registerSeal` call.

Step 1.5 (On-Chain Seal Registration). Any Ethereum account may call `registerSeal(target, sealData)` with $\text{sealData} = (r_s, s_s, \text{certId}, \text{envelopeHash})$. The registry re-checks the approval, re-validates the full trust chain, verifies V1 against the registered $(\text{cert.pk}_x, \text{cert.pk}_y)$, and atomically stores the seal record $(r_s, s_s, \text{certId}, \text{chainIdAtIssuance}, \text{envelopeHash})$ together with $\text{active} = \text{true}$. Authorisation is entirely cryptographic (V1 P-256 signature), not sender-based, enabling account abstraction and relayer submission.

C. Seal Verification

Step 2.1 (Seal Verification). Seal usage is read-only: V2 is verified once at certificate registration and V1 once at seal registration. Subsequent use therefore reduces to loading the stored seal and certificate records and re-evaluating dynamic

validity conditions. Algorithm 2 specifies this check. The only `Sealed(addr)` modifier exposed by `SealedContract` delegates to `SealCertificateRegistry.verifySeal`.

Algorithm 2 `verifySeal(target) / only-Sealed(counterparty)`

Require: Target contract address `target` with a registered seal record

Ensure: Returns true if and only if the registered seal is still active, its certificate's trust chain is intact, and its chain binding matches

Step 1: Load seal and verify chain binding

$\text{seal} \leftarrow \text{seals}[\text{target}]$
require $\text{seal.active} \wedge \text{seal.chainIdAtIssuance} = \text{block.chainId}$

Step 2: Load certificate and verify lifecycle

$\text{cert} \leftarrow \text{certificates}[\text{seal.certId}]$
require $\text{cert.active} \wedge \text{cert.pk}_x \neq 0$
require $\text{cert.t}_{\min} \leq \text{block.timestamp} \leq \text{cert.t}_{\max}$

Step 3: Re-check trust anchor

require `ISVALIDTRUSTSERVICE(cert.tsId)`
return true

No per-call P-256 verification is needed. A call succeeds if the stored seal is active, was issued for the current chain, and references a certificate that is still active, time-valid, and anchored in a currently valid trust service. Revocation may invalidate either the seal or the certificate. Hence verification is a pure state check.

D. On-Chain ASN.1/DER Parser

The on-chain parser closes the gap between “the QTSP signed a hash” and “the signed bytes contain the claimed identity fields.” During certificate registration, the contract extracts the public key, LPID, validity window, and serial directly from the submitted `TBSCertificate`, i.e., from the same byte string whose SHA-256 digest is verified under the QTSP issuer signature.

The parser performs a single linear walk over the `TBSCertificate` and extracts:

- the P-256 public key from `SubjectPublicKeyInfo`;
- the LPID from `organizationIdentifier` (OID 2.5.4.97), validated against the LEIXG- prefix;
- the validity window; and
- the certificate serial for CRL-based revocation.

The parser is intentionally narrow: it accepts only the eIDAS `QSeal` profile used in our protocol and reverts on malformed or unexpected encodings. We do not claim a formal proof of parser correctness and treat the parser as part of the trusted computing base.

E. Revocation

We define three revocation paths to clear or deactivate a seal: (i) the Contract Owner removes its own seal via `removeSeal`, (ii) the Certificate Holder signs a revocation payload and submits it via `revokeSealByIssuer`, which

verifies the P-256 signature and clears the seal, and (iii) any party submits a DER-encoded CRL signed by the SDI, whose serial listing `CRLParser` searches to permanently set `cert.active = false`. OCSP and CRL Distribution Points are universally deployed by EU QTSPs, so coverage is preserved. The issuer revocation payload binds (target, cid, certId) without a nonce. Post-revocation replay is defeated by the standard eIDAS lifecycle of certificate rotation.

VI. DISCUSSION

Registration-once, verify-always. The steady-state verification path performs *zero* P-256 operations: both signatures are verified exactly once, V2 at `registerCertificate` and V1 at `registerSeal`, and cached as unforgeable on-chain state. Monotonic deactivation of the stored seal flag guarantees it cannot be re-established without a fresh V1 check, so expensive cryptographic work is incurred once rather than by every downstream verifier.

Deferred verification as a reusable pattern. The two-layer caching used here is not specific to eIDAS. Any protocol whose trust artifact is signed by a slowly changing authority and consumed repeatedly by many parties can follow the same pattern: verify once at registration, persist the result as monotone state, and reduce the steady-state path to bounded storage reads. The pattern shifts verification cost from the verifier to the registrant and decouples the hot path from signature-scheme cost, which is particularly valuable on platforms where new curves arrive only through protocol upgrades.

Generalisation beyond QSeal. Under eIDAS, Qualified Certificates for Electronic Signatures (QES) and for Website Authentication (QWAC) share the same profile family and the same QTSP-to-LOTL trust chain as QSealC. The identity fields (public key, LPID for QSeal and QWAC, natural-person identifier for QES) occupy the same positions within the TBS, so the on-chain DER parser extends to QES and QWAC without protocol-level changes. KYC Seal is therefore a *template for all qualified trust services on the EVM*. The principal obstacle for QES is regulatory rather than cryptographic.

Governance of the on-chain trust root. Mirroring the LOTL onto the chain is the last remaining trust assumption and the main attack surface. Four governance designs trade off differently. (i) *Admin-curated mirror* (the current prototype), analogous to browser CA stores: simple and auditable, but (semi) centralised. (ii) *Protocol-level trust anchor*, with the LOTL embedded in the Ethereum client in the same way browsers ship CA stores: strongest integrity, but tightly coupled to client releases. (iii) *Individual trusted list*, where on-chain actors manually select QTSPs and pre-approve them as root trust sources: practical, with the added overhead of individually curating lists and comparing them to counterparties. (iv) *EU-operated on-chain publication*, in which the European Commission publishes the authoritative trust-service set directly on-chain: the on-chain list becomes the primary source and the web LOTL a legacy mirror, potentially raising trust *above* the current web-served distribution by substituting consensus

finality for HTTPS-transport assumptions. Regardless of the chosen option, the governance surface should be hardened with transparency around root updates, append-only audit logs of trust-list mutations, delayed activation of new trust anchors, and multi-party governance with threshold approval for additions and removals.

Limitations. (i) The DER parser is narrow by construction and not formally verified, and is therefore treated as part of the trusted computing base. (ii) The curated trust-root assumption reduces on-chain verification to “governance of the on-chain LOTL mirror is honest,” paralleling rather than eliminating the trust placed in browser CA stores. (iii) The seal binds a single CAdES envelope; legitimate envelope rotation (for example, BES $\rightarrow$ LT) requires a fresh seal signature and a new `registerSeal` call.

Future work. (i) A formally verified DER parser, produced through a proof-carrying ASN.1 toolchain, would remove the parser from the trusted computing base. (ii) Seal portability across chains would generalise the current per-chain-ID binding into a signed portability statement, enabling a single legal entity to maintain consistent identity across L1 and L2 without independent re-sealing on each chain. (iii) Natural-person extensions (QES) with selective disclosure or zero-knowledge proofs would bring GDPR-sensitive attribution within reach of the same architecture. (iv) Standardisation via EU initiatives or ERC proposals.

VII. CONCLUSION

We presented *KYC Seal*, the first protocol that verifies the full EU eIDAS trust chain (from the European Commission’s LOTL through Member-State TSLs and QTSP certificates down to an individual smart contract) natively inside EVM execution, closing the attribution gap that has blocked compliance with PSD2/PSR, AMLR, and MiCA on public blockchains. Per-interaction seal verification reduces to a pure on-chain state check, while the one-time registration steps perform the QTSP issuer-signature verification and the on-chain DER parse of the TBS, made economical by the recent P-256 precompile on the EVM. The on-chain ASN.1/DER parser, the registration-time caching of both verifications, and the trust-list governance model are protocol-level components that generalise beyond QSeal to the wider family of eIDAS qualified trust services. A reference implementation, a formal security analysis, and a gas evaluation are the subject of forthcoming work. We invite ERC-level standardisation of the on-chain trust-list semantics, certificate-record layout, and seal-verification interface exposed by this work.

REFERENCES

- [1] OECD, “The tokenisation of assets and potential implications for financial markets,” OECD Publishing, Paris, Tech. Rep., 2020.
- [2] A. Vaziry, K. Barman, and P. Herbke, “SoK: Bridging Trust into the Blockchain. A Systematic Review on On-Chain Identity,” in *2024 6th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)*. Berlin, Germany: IEEE, Oct. 2024, pp. 1–9.

- [3] C. Wronka, "Crypto-asset activities and markets in the european union: issues, challenges and considerations for regulation, supervision and oversight," *Journal of Banking Regulation*, vol. 25, no. 1, pp. 84–93, 2024.
- [4] C. Beekhuizen, U. Erdoğan, and D. Alpaslan, "EIP-7951: Precompile for secp256r1 Curve Support," Ethereum Improvement Proposals, no. 7951, 2025, status: Final. Accessed: 2026-04-21. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-7951>
- [5] S. Matsumoto and R. M. Reischuk, "IKP: Turning a PKI around with decentralized automated incentives," in *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2017, pp. 410–426.
- [6] M. Y. Kubilay, M. S. Kiraz, and H. A. Mantar, "CertLedger: A new PKI model with certificate transparency based on blockchain," *Computers & Security*, vol. 85, pp. 333–352, 2019.
- [7] Z. Wang, J. Lin, Q. Cai, Q. Wang, D. Zha, and J. Jing, "Blockchain-based certificate transparency and revocation transparency," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 1, pp. 681–697, 2022.
- [8] M. Al-Bassam, "SCPki: A smart contract-based PKI and identity system," in *Proceedings of the ACM Workshop on Blockchain, Cryptocurrencies and Contracts (BCC '17)*. ACM, 2017.
- [9] C. Patsonakis, K. Samari, A. Kiayias, and M. Roussopoulos, "Implementing a smart contract PKI," *IEEE Transactions on Engineering Management*, vol. 67, no. 4, pp. 1425–1443, 2020.
- [10] T. Saleem, M. U. Janjua, M. Hassan, T. Ahmad, F. Tariq, K. Hafeez, M. A. Salal, and M. D. Bilal, "ProofChain: An X.509-compatible blockchain-based PKI framework with decentralized trust," *Computer Networks*, vol. 213, p. 109069, 2022.
- [11] A. Yakubov, W. M. Shbair, A. Wallbom, D. Sanda, and R. State, "A blockchain-based PKI management framework," in *2018 IEEE/IFIP Network Operations and Management Symposium (NOMS)*. IEEE, 2018.
- [12] Y. Bak, "zk-X509: Privacy-preserving on-chain identity from legacy PKI via zero-knowledge proofs," 2026.
- [13] S. Khan *et al.*, "A survey on X.509 public-key infrastructure, certificate revocation, and their modern implementation on blockchain and ledger technologies," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, 2023.
- [14] WebOfTrust, "vlei: Public open specifications for gleif vlei (verifiable legal entity identifiers)," <https://github.com/WebOfTrust/vLEI>, accessed: 2025-12-16.
- [15] G. Bartolomeo, "Functional credentials: A practical construction for the European digital identity," in *Computer Security – ESORICS 2025*, ser. Lecture Notes in Computer Science, vol. 16056. Springer, 2025.
- [16] S. Gómez, "European Trust Model based on EBSI and Reusable eIDAS Attestations," Gataca Labs S.L.U., Technical Report v1.0, Apr. 2025. [Online]. Available: https://blockstand.eu/blockstand/uploads/2025/05/European-Trust-Model_-EBSI-and-Reusable-eIDAS-Attestations-1.pdf
- [17] I. Alamillo, S. Schwalm, C. Stoecker, and R. Thiermann, "Qualified Ledgers – Breakthrough for proven security and legal trust in DLT through eIDAS2 Regulation?" in *Open Identity Summit 2024*, ser. Lecture Notes in Informatics (LNI), vol. P-350. Bonn: Gesellschaft für Informatik e.V., 2024, pp. 195–200.
- [18] European Parliament and Council of the European Union, "REGULATION (EU) No 910/2014 of the European Parliament and of the Council of 23 July 2014 on electronic identification and trust services for electronic transactions in the internal market and repealing Directive 1999/93/EC," Jul. 2014. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2014/910/oj>
- [19] European Parliament and Council of the European Union, "REGULATION (EU) 2024/1183 of the European Parliament and of the Council of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework," Apr. 2024. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2024/1183/oj>
- [20] European Commission, "Commission Implementing Regulation (EU) 2024/2979 of 28 November 2024 laying down rules for the application of Regulation (EU) No 910/2014 of the European Parliament and of the Council as regards the integrity and core functionalities of European Digital Identity Wallets," Nov. 2024. [Online]. Available: https://eur-lex.europa.eu/eli/reg_impl/2024/2979/oj
- [21] ETSI, "TS 119 312 - V1.5.1 - Electronic Signatures and Trust Infrastructures (ESI); Cryptographic Suites," 2024. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/119300_119399/119312/01.05.01_60/ts_119312v010501p.pdf
- [22] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," RFC 5280, Internet Engineering Task Force, May 2008. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5280>
- [23] ETSI, "ETSI EN 319 412-1 – Certificate Profiles; Part 1: Overview and common data structures," 2023. [Online]. Available: https://www.etsi.org/deliver/etsi_en/319400_319499/31941201/01.05.01_60/en_31941201v010501p.pdf
- [24] European Parliament and Council of the European Union, "Directive (eu) 2015/2366 of the european parliament and of the council of 25 november 2015 on payment services in the internal market (psd2)," pp. 35–127, Nov. 2015. [Online]. Available: <https://eur-lex.europa.eu/eli/dir/2015/2366/oj>
- [25] European Commission, "Proposal for a regulation on payment services in the internal market (psr)," EUR-Lex, Jun. 2023. [Online]. Available: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=celex:52023PC0367>
- [26] European Parliament and Council of the European Union, "Regulation (eu) 2024/1624 of the european parliament and of the council of 31 may 2024 on the prevention of the use of the financial system for the purposes of money laundering or terrorist financing," May 2024. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2024/1624/oj>
- [27] European Parliament and Council of the European Union, "Regulation (eu) 2023/1114 of the european parliament and of the council of 31 may 2023 on markets in crypto-assets (mica)," pp. 40–205, Jun. 2023. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2023/1114/oj>
- [28] ETSI, "ETSI EN 319 122-1 V1.3.1 – Electronic Signatures and Infrastructures (ESI); CADES digital signatures; Part 1: Building blocks and CADES baseline signatures," 2023. [Online]. Available: https://www.etsi.org/deliver/etsi_en/319100_319199/31912201/01.03.01_60/en_31912201v010301p.pdf